VRUSHABH SAWANT SYVLSI_44 125BTVL1033

Experiment 9

Academic Year 2026-2027			
Program: Electronics Engineering (VLSI Design and Technology)			
Year of Study	Second Year	**Semester**	III
Course	Skill Lab: Python Programming	**Course Code**	VDCOR2VS201
Class	SY VLSI	**Course In-Charge**	Mr. Nirmol Munvar
Exp Name	Choose and apply Library for Appropriate Application - I		
CO	Evaluate between various Python libraries such as NumPy, pandas, and matplotlib to process, analyze, and visualize data for engineering application		

Python Standard and Commonly Used Libraries

Python provides a large collection of libraries containing pre-written functions, classes, and tools. Libraries allow programmers to solve common problems without implementing every operation from scratch.

A library can be imported using the import statement:

```
import random
```

Specific functions or classes can also be imported:

```
from datetime import datetime
```

The following libraries are useful for general programming, data processing, automation, numerical computation, and VLSI-related programming tasks.

OS Library

The os library provides functions for interacting with the **operating system**. It is commonly used for working with files, directories, paths, and environment variables.

```
import os
```

os.getcwd() returns the current working directory.

```
print(os.getcwd())
```

os.listdir() returns the files and directories present in a location.

```
print(os.listdir())
```

os.mkdir() creates a new directory.

```
os.mkdir("test_data")
```

os.path.exists() checks whether a file or directory exists.

```
if os.path.exists("data.txt"):
    print("File exists")
```

os.path.join() constructs a file path in an operating-system-independent way.

```
path = os.path.join("data", "results.txt")
print(path)
```

The os library is useful when a program needs to interact with the **file system or operating system**.

random Library

The random library is used to generate **pseudo-random values** and make random selections.

```
import random
```

random.randint() generates a random integer within the specified range.

```
number = random.randint(1, 100)
```

print(number)
random.random() generates a random floating-point number between 0 and 1.
value = random.random()
print(value)
random.choice() randomly selects one element from a sequence.
signals = ["clk", "reset", "data", "valid"]

print(random.choice(signals))
random.shuffle() randomly rearranges the elements of a list.
numbers = [1, 2, 3, 4, 5]

random.shuffle(numbers)

print(numbers)
The random library is useful for **simulation, games, randomized testing, test-data generation, and verification scenarios**.

math Library

The math library provides mathematical functions and constants that are not available directly through Python's basic arithmetic operators.
import math
math.sqrt() calculates the square root.
print(math.sqrt(25))
math.pow() calculates a number raised to a power.
print(math.pow(2, 3))
math.ceil() rounds a number upward.
print(math.ceil(4.2))
math.floor() rounds a number downward.
print(math.floor(4.8))
The library also provides mathematical constants such as math.pi.
print(math.pi)
Other useful functions include:
math.sin()
math.cos()
math.tan()
math.log()
math.factorial()
math.gcd()
For example:
angle = math.radians(30)

print(math.sin(angle))
The math library is useful for **engineering calculations, geometry, trigonometry, numerical algorithms, and mathematical processing**.

NumPy Library

NumPy is a widely used library for numerical and scientific computing. It provides efficient multidimensional arrays and mathematical operations.
It is installed separately in most Python environments.
import numpy as np
A NumPy array can be created using:
numbers = np.array([1, 2, 3, 4, 5])

NumPy supports efficient element-wise operations.

```
numbers = np.array([1, 2, 3, 4])

print(numbers * 2)
```

Output:

```
[2 4 6 8]
```

NumPy provides functions for creating arrays:

```
zeros = np.zeros(5)
ones = np.ones(5)
values = np.arange(1, 10, 2)
```

It can also perform mathematical operations on complete arrays.

```
numbers = np.array([10, 20, 30, 40])

print(np.mean(numbers))
print(np.max(numbers))
print(np.min(numbers))
print(np.sum(numbers))
```

NumPy is particularly useful for **numerical data, matrices, vectors, signal processing, scientific computing, and engineering calculations**.

A useful distinction is that math primarily operates on individual numerical values, while NumPy is designed for efficient operations on **arrays and multidimensional numerical data**.

re Library

The re library provides support for **regular expressions**. Regular expressions are patterns used to search, match, extract, validate, and replace text.

```
import re
```

re.search() searches for a pattern within a string.

```
text = "Python programming"

result = re.search("Python", text)

if result:
    print("Pattern found")
```

re.findall() finds all occurrences that match a pattern.

```
text = "The addresses are 100, 250 and 400"

numbers = re.findall(r"\d+", text)

print(numbers)
```

Output:

```
['100', '250', '400']
```

re.sub() replaces text matching a pattern.

```
text = "VLSI is interesting"

new_text = re.sub("interesting", "important", text)

print(new_text)
```

Regular expressions are particularly useful for **log-file analysis, extracting information from text, validating input, and identifying patterns in source files or verification logs**.

For example, a regular expression can be used to extract hexadecimal addresses from a verification log:

```
text = "Address mismatch at 0x1A2F"

address = re.findall(r"0x[0-9A-Fa-f]+", text)

print(address)
```

datetime Library

The datetime library provides classes and functions for working with **dates and times**.

```
from datetime import datetime, date, timedelta
```

The current date and time can be obtained using:

```
now = datetime.now()

print(now)
```

The current date can be obtained using:

```
today = date.today()

print(today)
```

A specific date can be created using:

```
d = date(2026, 8, 12)

print(d)
```

Dates can be compared:

```
date1 = date(2026, 8, 10)
date2 = date(2026, 8, 12)

if date1 < date2:
    print("date1 is earlier")
```

timedelta can be used for date and time calculations.

```
today = date.today()

next_week = today + timedelta(days=7)

print(next_week)
```

The datetime library is useful for **timestamps, scheduling, logging, time calculations, and processing date-based data**.

itertools Library

The itertools library provides tools for working efficiently with **iterators and combinations of data**.

```
import itertools
```

itertools.permutations() generates possible arrangements of elements.

```
items = [1, 2, 3]

for p in itertools.permutations(items):
    print(p)
```

itertools.combinations() generates combinations where the order does not matter.

```
items = [1, 2, 3, 4]

for c in itertools.combinations(items, 2):
    print(c)
```

itertools.product() generates the Cartesian product of multiple iterables.

```
a = [1, 2]
b = ["A", "B"]

for item in itertools.product(a, b):
    print(item)
```

itertools.chain() allows multiple iterables to be processed as one sequence.

```
a = [1, 2]
b = [3, 4]

for x in itertools.chain(a, b):
    print(x)
```

itertools is useful for **test-case generation, combinations, permutations, Cartesian products, and complex iteration problems**.

copy Library

The copy library provides mechanisms for copying Python objects.

```
import copy
```

There are two important types of copying: **shallow copy** and **deep copy**.

A shallow copy creates a new outer object, but nested objects may still be shared.

```
original = [[1, 2], [3, 4]]

new_list = copy.copy(original)
```

A deep copy creates an independent copy, including nested objects.

```
original = [[1, 2], [3, 4]]

new_list = copy.deepcopy(original)

new_list[0][0] = 100

print(original)
print(new_list)
```

Output:

```
[[1, 2], [3, 4]]
[[100, 2], [3, 4]]
```

The original remains unchanged because deepcopy() creates independent copies of the nested objects.

The copy library is useful when working with **nested lists, dictionaries, objects, configurations, and other mutable data structures**.

collections Library

The collections library provides specialized container data types that extend Python's built-in data structures.

```
import collections
```

Counter

Counter is used to count how frequently elements occur.

```
from collections import Counter

signals = ["clk", "data", "clk", "reset", "data", "clk"]

count = Counter(signals)
```

```
print(count)
```
Output:
```
Counter({'clk': 3, 'data': 2, 'reset': 1})
```
This is useful for **frequency analysis and counting repeated values**.

defaultdict

defaultdict provides a default value when a key does not exist.
```
from collections import defaultdict

marks = defaultdict(list)

marks["VLSI"].append(85)
marks["VLSI"].append(90)

print(marks)
```
This is useful when data needs to be grouped by keys.

deque

deque is a double-ended queue that supports efficient insertion and removal from both ends.
```
from collections import deque

q = deque([1, 2, 3])

q.append(4)
q.appendleft(0)

print(q)
```
The collections library is useful when standard list and dict structures do not provide the most suitable behavior for a particular problem.

Basic Exercises

The following exercises are intentionally designed so that students must **analyze the problem and decide which library or combination of libraries would be appropriate**. The library name should not be specified in the question.

Exercise 1: VLSI Test Data Generator

You are developing a verification environment for a memory design. Generate **100 random memory addresses** in the range 0x0000 to 0x0FFF.

The program should:

- Identify duplicate addresses.
- Display the frequency of each repeated address.
- Randomly select 10 addresses for additional testing.
- Create a directory called test_data and save the generated addresses in a file inside it.

Determine which Python library or combination of libraries can simplify this problem. Explain your choice.

Exercise 2: VLSI Signal Analysis

A file contains 1000 sampled voltage values of a digital signal.

Write a program that:

- Reads the values from the file.
- Converts them into a suitable numerical data structure.
- Calculates the minimum, maximum, mean, and standard deviation.
- Calculates the RMS value of the signal.
- Identifies values outside the range 0 V to 1.2 V.
- Stores the invalid samples in a separate file.

Determine which library or combination of libraries would be most appropriate for this problem and justify your choice.

Exercise 3: Verification Log Analyzer

A verification log contains messages such as:

[2026-08-12 10:15:21] INFO: Test started
[2026-08-12 10:15:25] ERROR: Address mismatch at 0x1A2F
[2026-08-12 10:15:28] WARNING: Timeout
[2026-08-12 10:15:30] ERROR: Data mismatch at 0x2B10

Write a program that:

- Extracts all timestamps.
- Extracts all hexadecimal addresses.
- Extracts all ERROR messages.
- Counts the number of INFO, WARNING, and ERROR messages.
- Determines the earliest and latest timestamp.
- Displays a summary of the verification run.

Determine which libraries are useful for **pattern extraction, counting, and date/time processing**.

Exercise 4: VLSI Test Scenario Generator

A verification engineer wants to test a design using:

- 3 clock frequencies
- 4 voltage levels
- 2 operating modes
- 3 temperature conditions

Generate every possible test configuration.

Then:

- Determine the total number of possible configurations.
- Randomly select 10 configurations for immediate testing.
- Store the selected configurations.
- Create an independent copy of the selected configurations and modify the copy without affecting the original.

Determine which library or combination of libraries can simplify the generation, random selection, and copying of the test scenarios.

Exercise 5: Signal and Port Analysis

Given the following list:

```
signals = [
    "clk",
    "reset",
    "data_in",
    "data_out",
    "addr",
    "addr",
    "clk",
    "valid",
    "ready"
]
```

Write a program that:

- Finds the frequency of every signal.
- Identifies duplicate signal names.
- Extracts all signals containing "data".
- Randomly selects three signals for a test.

- Creates an independent copy of the signal list before modifying it.
- Calculates the total number of unique signals.

Determine which library or combination of libraries would be most suitable for each operation.

Exercise 6: Engineering Calculation and Test Generation

A VLSI engineer wants to generate test values for a circuit operating at different frequencies and calculate the corresponding time periods.

Given a list of frequencies in MHz:

frequencies = [10, 25, 50, 100, 200]

Write a program that:

- Calculates the time period corresponding to each frequency.
- Determines the minimum and maximum frequency.
- Generates all possible pairs of frequencies for comparison.
- Randomly selects three pairs for testing.
- Stores the generated test data in a separate structure.

Students should determine whether **one library or multiple libraries** are appropriate for the mathematical calculation, pair generation, and random selection.

Exercise 7: Test Result Frequency Analysis

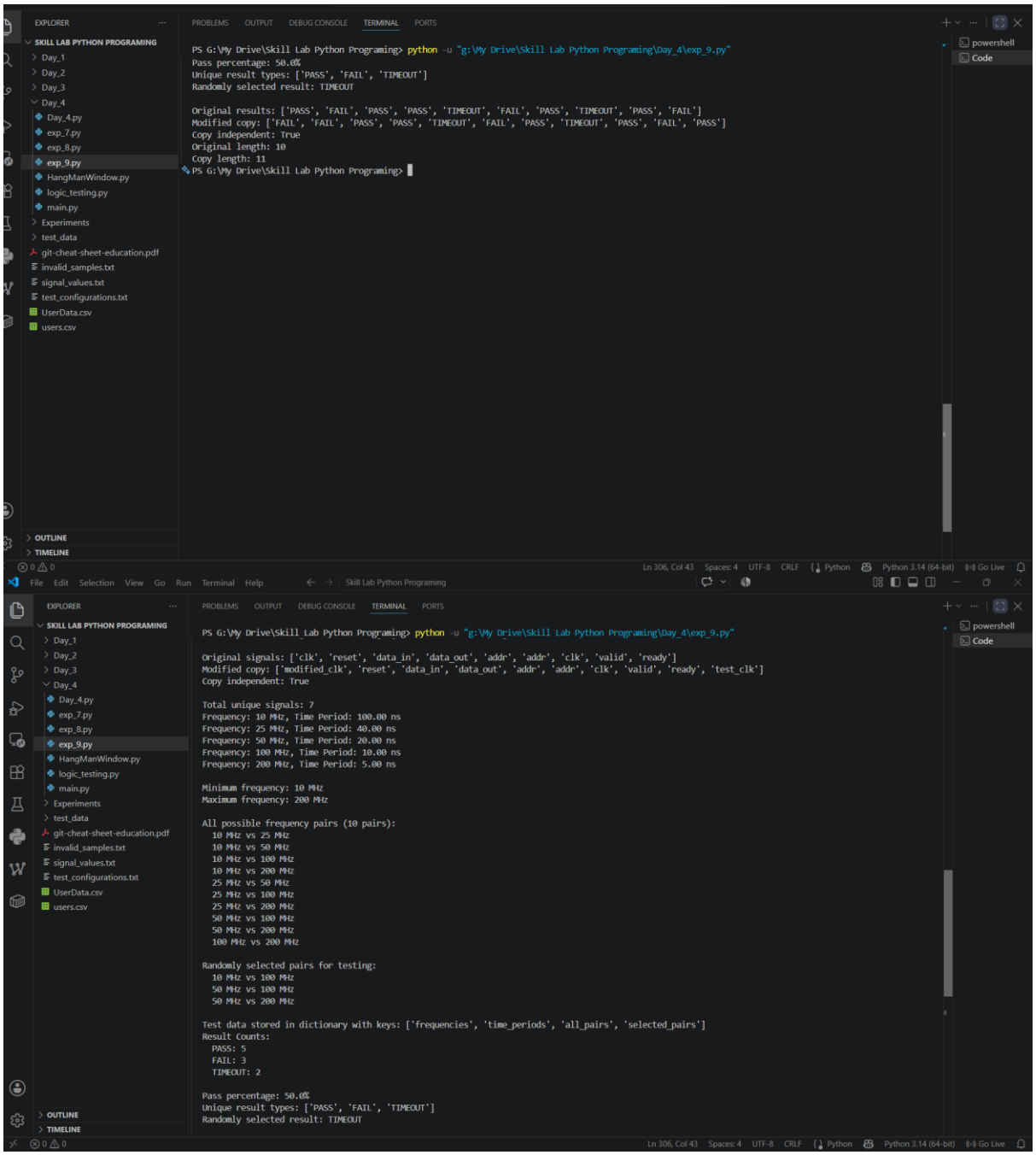
A verification program produces the following results:

```
results = [
   "PASS", "FAIL", "PASS", "PASS",
   "TIMEOUT", "FAIL", "PASS", "TIMEOUT",
   "PASS", "FAIL"
]
```

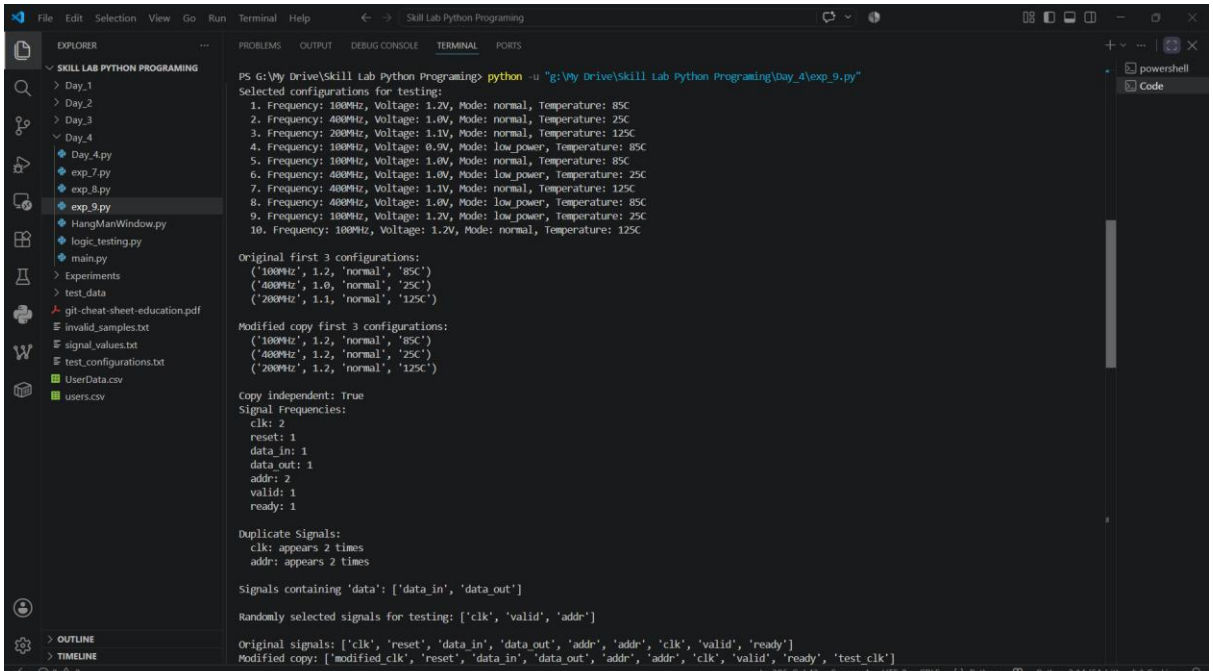
Write a program that:

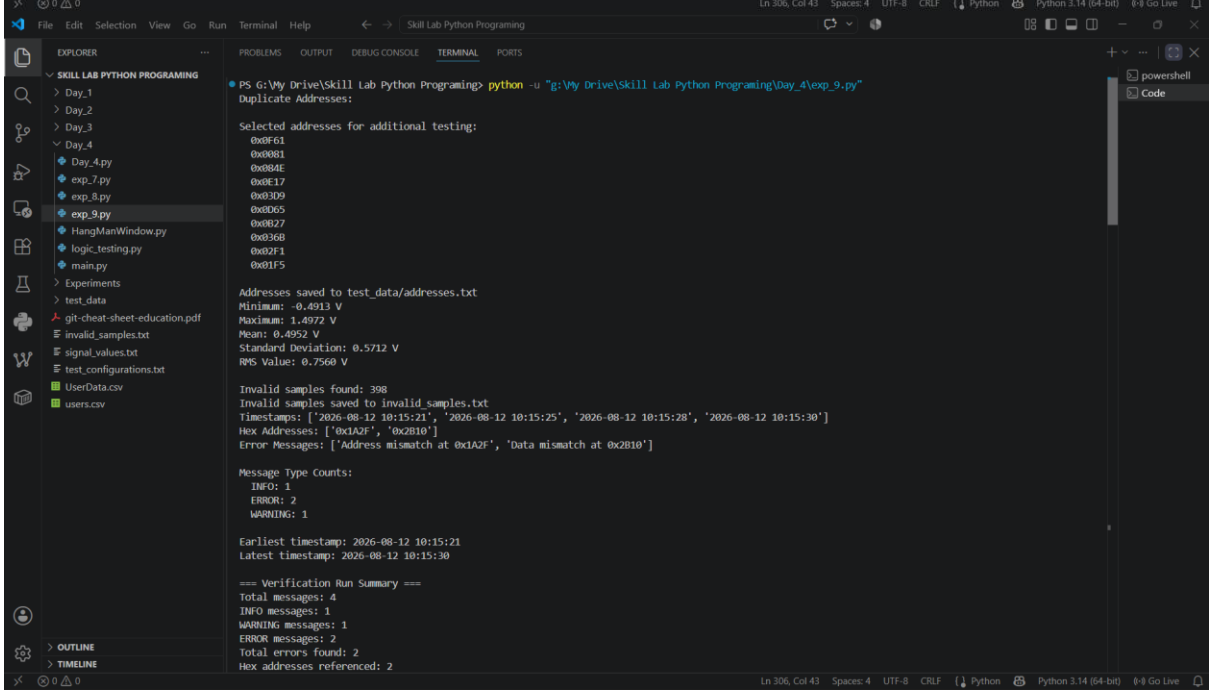
- Counts the occurrence of each result type.
- Calculates the percentage of tests that passed.
- Identifies all unique result types.
- Randomly selects one result for further analysis.
- Creates a copy of the result list and modifies the copy without modifying the original.

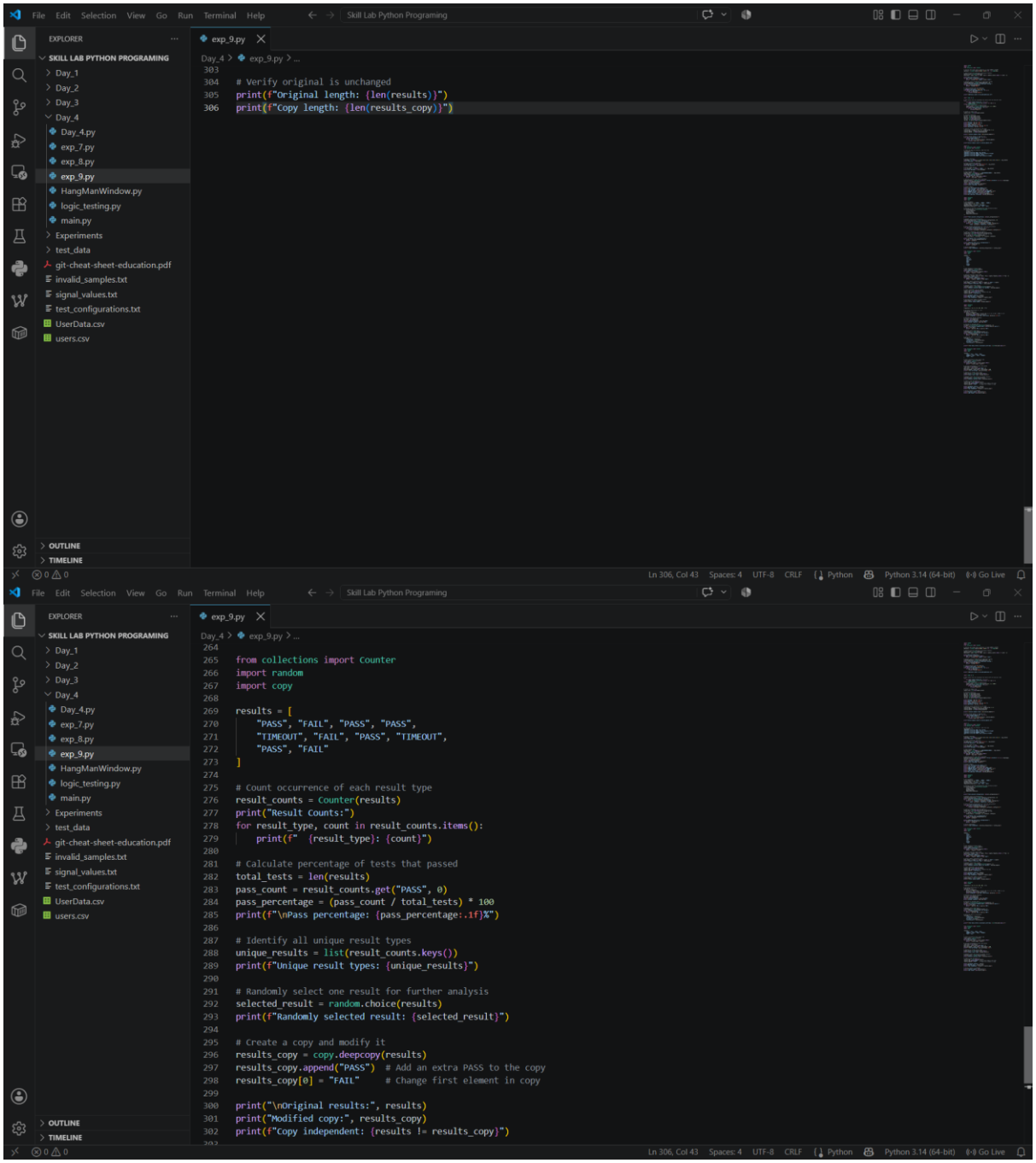
Determine which Python libraries or built-in features would be useful and explain why.


EXPLORER
SKILL LAB PYTHON PROGRAMING
Day_1
Day_2
Day_3
Day_4
Day_4.py
exp_7.py
exp_8.py
exp_9.py
HangManWindow.py
logic_testing.py
main.py
Experiments
test_data
git-cheat-sheet-education.pdf
invalid_samples.txt
signal_values.txt
test_configurations.txt
UserData.csv
users.csv
OUTLINE
TIMELINE
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
powershell
Code
PS G:\My Drive\Skill Lab Python Programing> python -u "g:\My Drive\Skill Lab Python Programing\Day_4\exp_9.py"
Pass percentage: 50.0%
Unique result types: ['PASS', 'FAIL', 'TIMEOUT']
Randomly selected result: TIMEOUT
Original results: ['PASS', 'FAIL', 'PASS', 'PASS', 'TIMEOUT', 'FAIL', 'PASS', 'TIMEOUT', 'PASS', 'FAIL']
Modified copy: ['FAIL', 'FAIL', 'PASS', 'PASS', 'TIMEOUT', 'FAIL', 'PASS', 'TIMEOUT', 'PASS', 'FAIL', 'PASS']
Copy independent: True
Original length: 10
Copy length: 11
PS G:\My Drive\Skill Lab Python Programing>
Ln 306, Col 43
Spaces: 4
UTF-8
CRLF
Python
Python 3.14 (64-bit)
Go Live

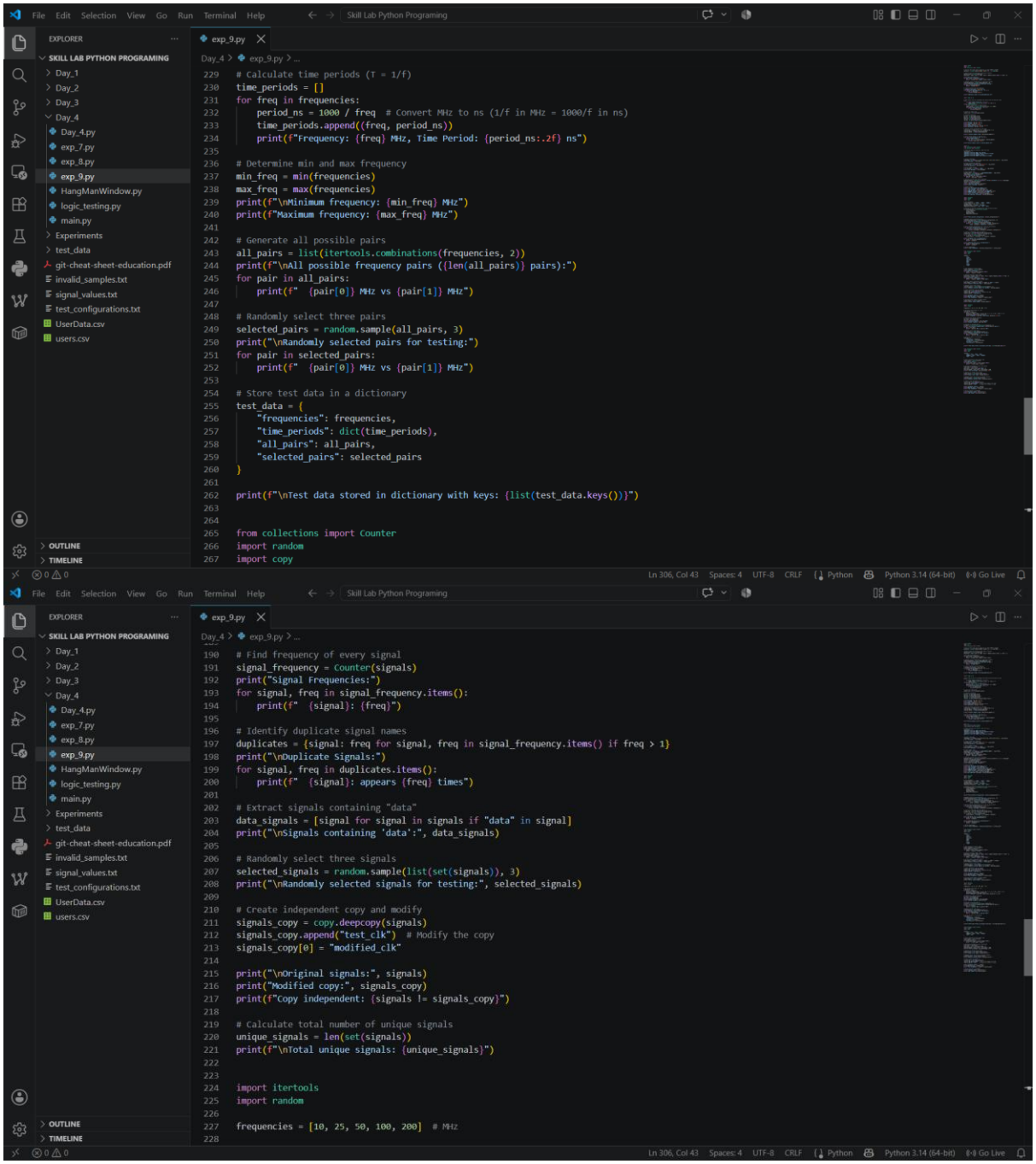
File Edit Selection View Go Run Terminal Help
Skill Lab Python Programing
EXPLORER
SKILL LAB PYTHON PROGRAMING
Day_1
Day_2
Day_3
Day_4
Day_4.py
exp_7.py
exp_8.py
exp_9.py
HangManWindow.py
logic_testing.py
main.py
Experiments
test_data
git-cheat-sheet-education.pdf
invalid_samples.txt
signal_values.txt
test_configurations.txt
UserData.csv
users.csv
OUTLINE
TIMELINE
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
powershell
Code
PS G:\My Drive\Skill Lab Python Programing> python -u "g:\My Drive\Skill Lab Python Programing\Day_4\exp_9.py"
Original signals: ['clk', 'reset', 'data_in', 'data_out', 'addr', 'addr', 'clk', 'valid', 'ready']
Modified copy: ['modified_clk', 'reset', 'data_in', 'data_out', 'addr', 'addr', 'clk', 'valid', 'ready', 'test_clk']
Copy independent: True
Total unique signals: 7
Frequency: 10 MHz, Time Period: 100.00 ns
Frequency: 25 MHz, Time Period: 40.00 ns
Frequency: 50 MHz, Time Period: 20.00 ns
Frequency: 100 MHz, Time Period: 10.00 ns
Frequency: 200 MHz, Time Period: 5.00 ns
Minimum frequency: 10 MHz
Maximum frequency: 200 MHz
All possible frequency pairs (10 pairs):
10 MHz vs 25 MHz
10 MHz vs 50 MHz
10 MHz vs 100 MHz
10 MHz vs 200 MHz
25 MHz vs 50 MHz
25 MHz vs 100 MHz
25 MHz vs 200 MHz
50 MHz vs 100 MHz
50 MHz vs 200 MHz
100 MHz vs 200 MHz
Randomly selected pairs for testing:
10 MHz vs 100 MHz
50 MHz vs 100 MHz
50 MHz vs 200 MHz
Test data stored in dictionary with keys: ['frequencies', 'time_periods', 'all_pairs', 'selected_pairs']
Result Counts:
PASS: 5
FAIL: 3
TIMEOUT: 2
Pass percentage: 50.0%
Unique result types: ['PASS', 'FAIL', 'TIMEOUT']
Randomly selected result: TIMEOUT
Ln 306, Col 43
Spaces: 4
UTF-8
CRLF
Python
Python 3.14 (64-bit)
Go Live


File Edit Selection View Go Run Terminal Help
Skill Lab Python Programing
EXPLORER
SKILL LAB PYTHON PROGRAMING
Day_1
Day_2
Day_3
Day_4
Day_4.py
exp_7.py
exp_8.py
exp_9.py
HangManWindow.py
logic_testing.py
main.py
Experiments
test_data
git-cheat-sheet-education.pdf
invalid_samples.txt
signal_values.txt
test_configurations.txt
UserData.csv
users.csv
OUTLINE
TIMELINE
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
powershell
Code
PS G:\My Drive\Skill Lab Python Programing> python -u "g:\My Drive\Skill Lab Python Programing\Day_4\exp_9.py"
Selected configurations for testing:
1. Frequency: 100MHz, Voltage: 1.2V, Mode: normal, Temperature: 85C
2. Frequency: 400MHz, Voltage: 1.0V, Mode: normal, Temperature: 25C
3. Frequency: 200MHz, Voltage: 1.1V, Mode: normal, Temperature: 125C
4. Frequency: 100MHz, Voltage: 0.9V, Mode: low_power, Temperature: 85C
5. Frequency: 100MHz, Voltage: 1.0V, Mode: normal, Temperature: 85C
6. Frequency: 400MHz, Voltage: 1.0V, Mode: low_power, Temperature: 25C
7. Frequency: 400MHz, Voltage: 1.1V, Mode: normal, Temperature: 125C
8. Frequency: 400MHz, Voltage: 1.0V, Mode: low_power, Temperature: 85C
9. Frequency: 100MHz, Voltage: 1.2V, Mode: low_power, Temperature: 25C
10. Frequency: 100MHz, Voltage: 1.2V, Mode: normal, Temperature: 125C
Original first 3 configurations:
('100MHz', 1.2, 'normal', '85C')
('400MHz', 1.0, 'normal', '25C')
('200MHz', 1.1, 'normal', '125C')
Modified copy first 3 configurations:
('100MHz', 1.2, 'normal', '85C')
('400MHz', 1.2, 'normal', '25C')
('200MHz', 1.2, 'normal', '125C')
Copy independent: True
Signal Frequencies:
clk: 2
reset: 1
data_in: 1
data_out: 1
addr: 2
valid: 1
ready: 1
Duplicate Signals:
clk: appears 2 times
addr: appears 2 times
Signals containing 'data': ['data_in', 'data_out']
Randomly selected signals for testing: ['clk', 'valid', 'addr']
Original signals: ['clk', 'reset', 'data_in', 'data_out', 'addr', 'addr', 'clk', 'valid', 'ready']
Modified copy: ['modified_clk', 'reset', 'data_in', 'data_out', 'addr', 'addr', 'clk', 'valid', 'ready', 'test_clk']
Ln 306, Col 43 Spaces: 4 UTF-8 CRLF Python Python 3.14 (64-bit) Go Live

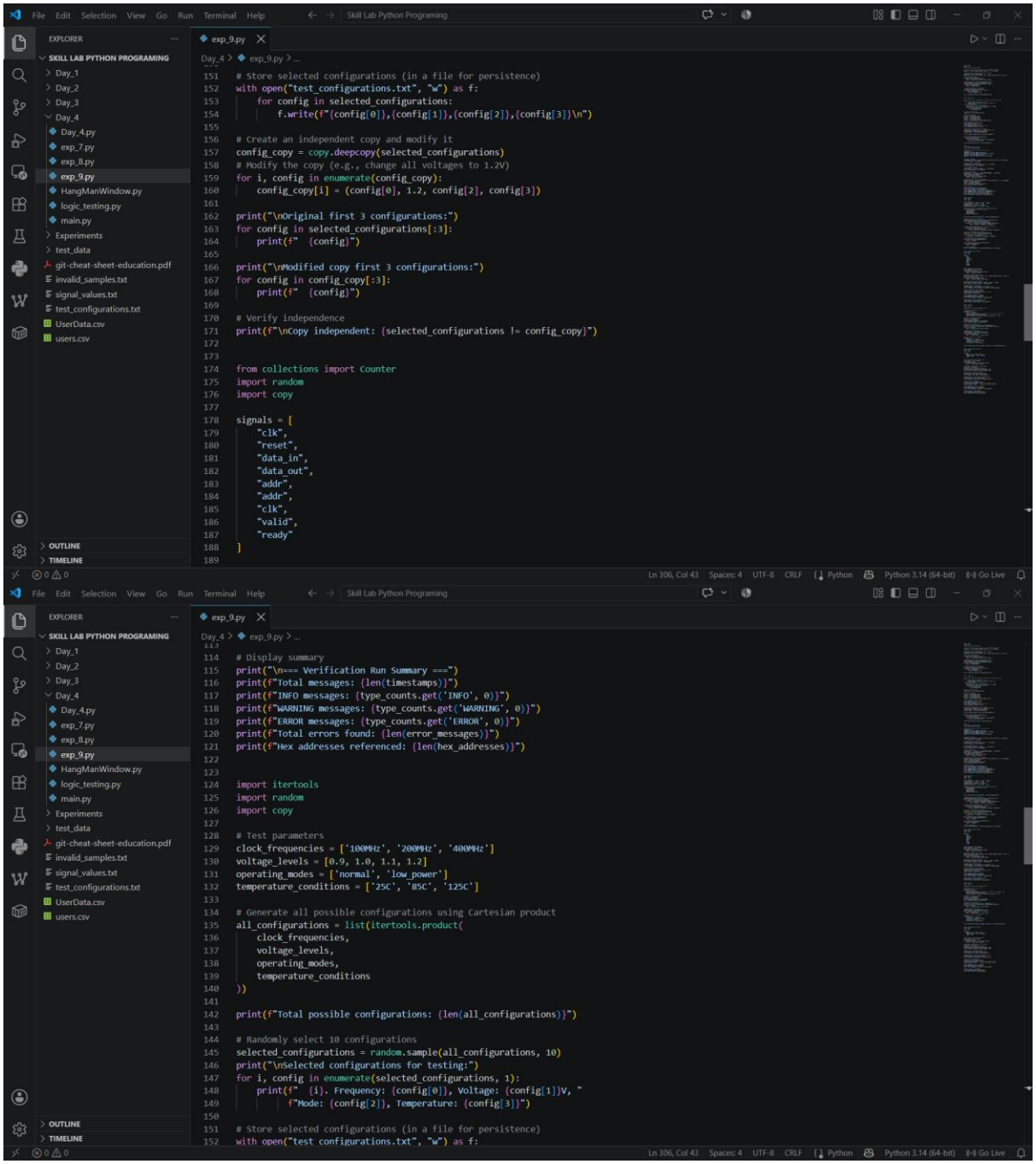

File Edit Selection View Go Run Terminal Help
Skill Lab Python Programing
EXPLORER
SKILL LAB PYTHON PROGRAMING
Day_1
Day_2
Day_3
Day_4
Day_4.py
exp_7.py
exp_8.py
exp_9.py
HangManWindow.py
logic_testing.py
main.py
Experiments
test_data
git-cheat-sheet-education.pdf
invalid_samples.txt
signal_values.txt
test_configurations.txt
UserData.csv
users.csv
OUTLINE
TIMELINE
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
powershell
Code
PS G:\My Drive\Skill Lab Python Programing> python -u "g:\My Drive\Skill Lab Python Programing\Day_4\exp_9.py"
Duplicate Addresses:
Selected addresses for additional testing:
0x0F61
0x0081
0x084E
0x0E17
0x03D9
0x0D65
0x0827
0x036B
0x02F1
0x01F5
Addresses saved to test_data/addresses.txt
Minimum: -0.4913 V
Maximum: 1.4972 V
Mean: 0.4952 V
Standard Deviation: 0.5712 V
RMS Value: 0.7560 V
Invalid samples found: 398
Invalid samples saved to invalid_samples.txt
Timestamps: ['2026-08-12 10:15:21', '2026-08-12 10:15:25', '2026-08-12 10:15:28', '2026-08-12 10:15:30']
Hex Addresses: ['0x1A2F', '0x2B10']
Error Messages: ['Address mismatch at 0x1A2F', 'Data mismatch at 0x2B10']
Message Type Counts:
INFO: 1
ERROR: 2
WARNING: 1
Earliest timestamp: 2026-08-12 10:15:21
Latest timestamp: 2026-08-12 10:15:30
=== Verification Run Summary ===
Total messages: 4
INFO messages: 1
WARNING messages: 1
ERROR messages: 2
Total errors found: 2
Hex addresses referenced: 2
Ln 306, Col 43 Spaces: 4 UTF-8 CRLF Python Python 3.14 (64-bit) Go Live


File Edit Selection View Go Run Terminal Help
Skill Lab Python Programing
EXPLORER
SKILL LAB PYTHON PROGRAMING
Day_1
Day_2
Day_3
Day_4
Day_4.py
exp_7.py
exp_8.py
exp_9.py
HangManWindow.py
logic_testing.py
main.py
Experiments
test_data
git-cheat-sheet-education.pdf
invalid_samples.txt
signal_values.txt
test_configurations.txt
UserData.csv
users.csv
OUTLINE
TIMELINE
exp_9.py
Day_4 > exp_9.py > ...
303
304 # Verify original is unchanged
305 print(f"Original length: {len(results)}")
306 print(f"Copy length: {len(results_copy)}")
Ln 306, Col 43 Spaces: 4 UTF-8 CRLF Python Python 3.14 (64-bit) Go Live

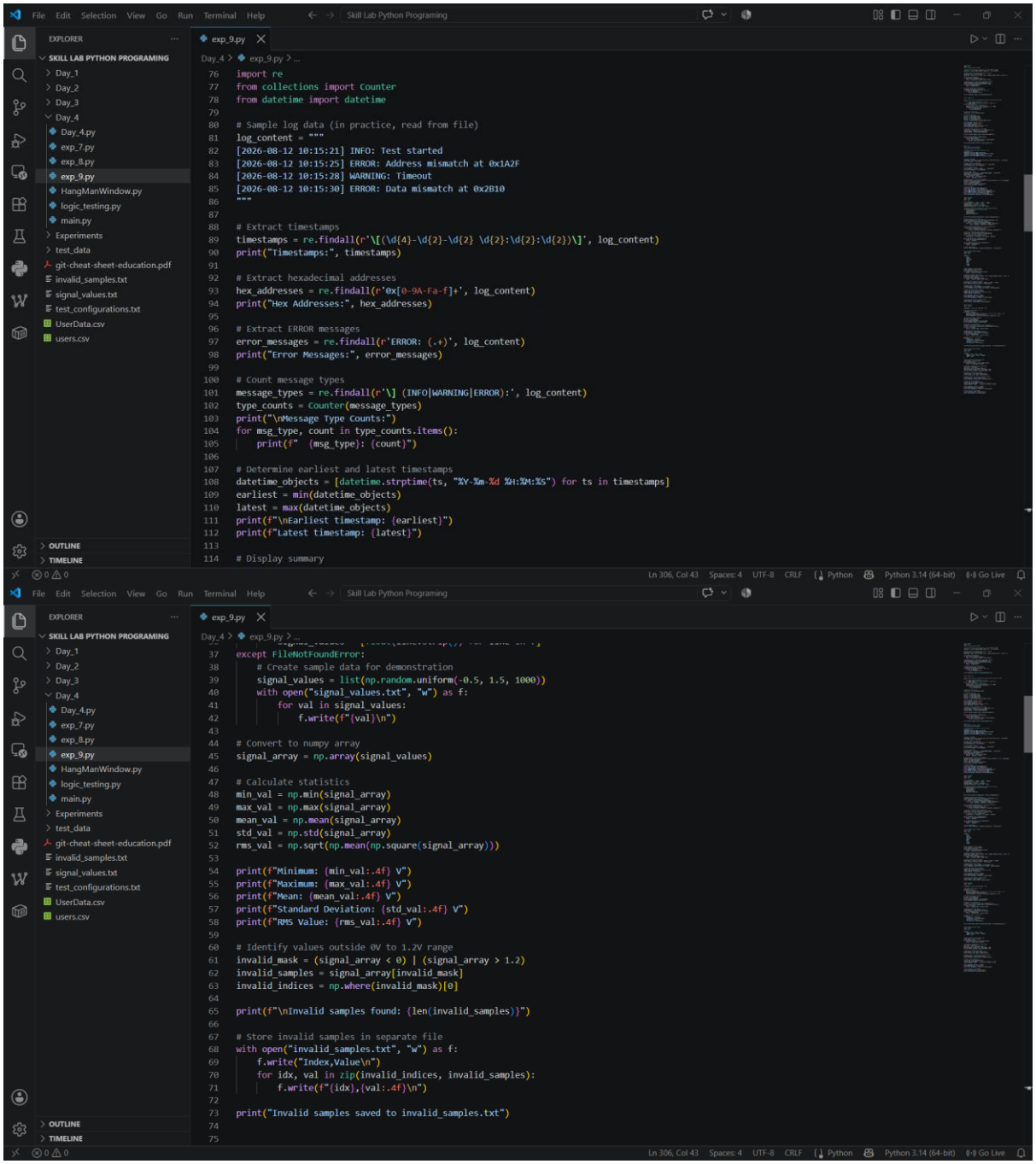
File Edit Selection View Go Run Terminal Help
Skill Lab Python Programing
EXPLORER
SKILL LAB PYTHON PROGRAMING
Day_1
Day_2
Day_3
Day_4
Day_4.py
exp_7.py
exp_8.py
exp_9.py
HangManWindow.py
logic_testing.py
main.py
Experiments
test_data
git-cheat-sheet-education.pdf
invalid_samples.txt
signal_values.txt
test_configurations.txt
UserData.csv
users.csv
OUTLINE
TIMELINE
exp_9.py
Day_4 > exp_9.py > ...
264
265 from collections import Counter
266 import random
267 import copy
268
269 results = [
270 "PASS", "FAIL", "PASS", "PASS",
271 "TIMEOUT", "FAIL", "PASS", "TIMEOUT",
272 "PASS", "FAIL"
273]
274
275 # Count occurrence of each result type
276 result_counts = Counter(results)
277 print("Result Counts:")
278 for result_type, count in result_counts.items():
279 print(f" {result_type}: {count}")
280
281 # Calculate percentage of tests that passed
282 total_tests = len(results)
283 pass_count = result_counts.get("PASS", 0)
284 pass_percentage = (pass_count / total_tests) * 100
285 print(f"\nPass percentage: {pass_percentage:.1f}%")
286
287 # Identify all unique result types
288 unique_results = list(result_counts.keys())
289 print(f"Unique result types: {unique_results}")
290
291 # Randomly select one result for further analysis
292 selected_result = random.choice(results)
293 print(f"Randomly selected result: {selected_result}")
294
295 # Create a copy and modify it
296 results_copy = copy.deepcopy(results)
297 results_copy.append("PASS") # Add an extra PASS to the copy
298 results_copy[0] = "FAIL" # Change first element in copy
299
300 print("\nOriginal results:", results)
301 print("Modified copy:", results_copy)
302 print(f"Copy independent: {results != results_copy}")
Ln 306, Col 43 Spaces: 4 UTF-8 CRLF Python Python 3.14 (64-bit) Go Live


File Edit Selection View Go Run Terminal Help
Skill Lab Python Programing
EXPLORER
SKILL LAB PYTHON PROGRAMING
Day_1
Day_2
Day_3
Day_4
Day_4.py
exp_7.py
exp_8.py
exp_9.py
HangManWindow.py
logic_testing.py
main.py
Experiments
test_data
git-cheat-sheet-education.pdf
invalid_samples.txt
signal_values.txt
test_configurations.txt
UserData.csv
users.csv
OUTLINE
TIMELINE
exp_9.py
Day_4 > exp_9.py > ...
# Calculate time periods (T = 1/f)
time_periods = []
for freq in frequencies:
period_ns = 1000 / freq # Convert MHz to ns (1/f in MHz = 1000/f in ns)
time_periods.append((freq, period_ns))
print(f"Frequency: {freq} MHz, Time Period: {period_ns:.2f} ns")
# Determine min and max frequency
min_freq = min(frequencies)
max_freq = max(frequencies)
print(f"\nMinimum frequency: {min_freq} MHz")
print(f"Maximum frequency: {max_freq} MHz")
# Generate all possible pairs
all_pairs = list(itertools.combinations(frequencies, 2))
print(f"\nAll possible frequency pairs ({len(all_pairs)} pairs):")
for pair in all_pairs:
print(f" {pair[0]} MHz vs {pair[1]} MHz")
# Randomly select three pairs
selected_pairs = random.sample(all_pairs, 3)
print("\nRandomly selected pairs for testing:")
for pair in selected_pairs:
print(f" {pair[0]} MHz vs {pair[1]} MHz")
# Store test data in a dictionary
test_data = {
"frequencies": frequencies,
"time_periods": dict(time_periods),
"all_pairs": all_pairs,
"selected_pairs": selected_pairs
}
print(f"\nTest data stored in dictionary with keys: {list(test_data.keys())}")
from collections import Counter
import random
import copy
Ln 306, Col 43 Spaces: 4 UTF-8 CRLF Python Python 3.14 (64-bit) Go Live

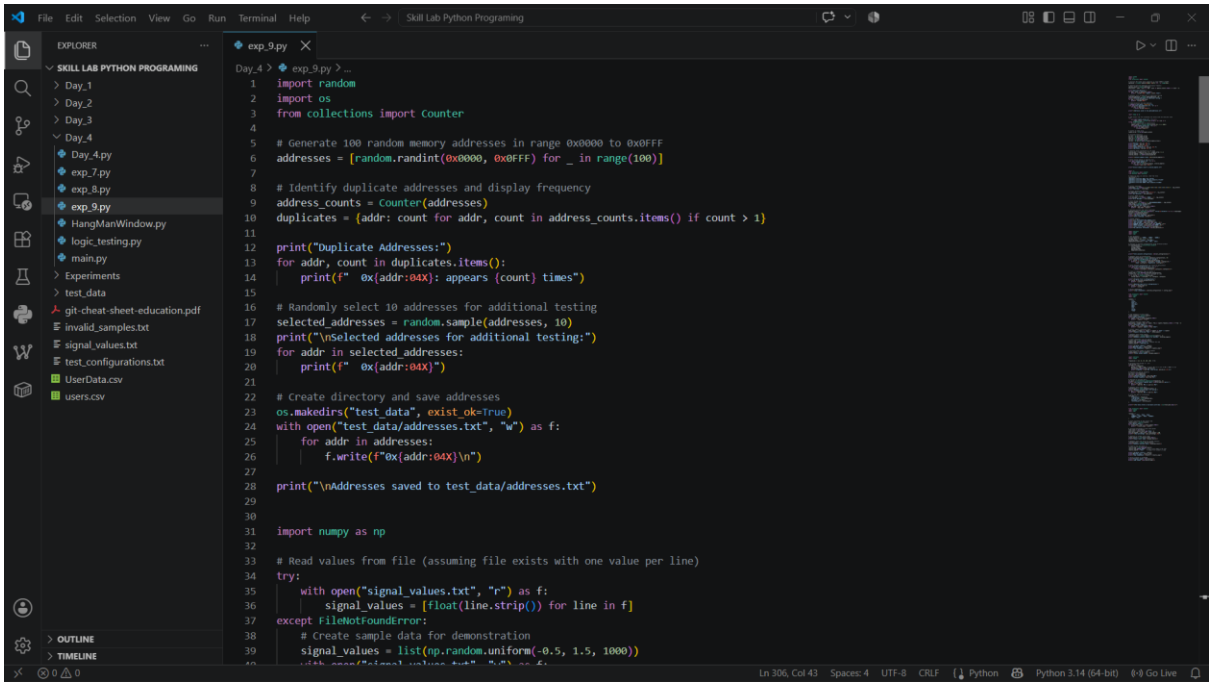
File Edit Selection View Go Run Terminal Help
Skill Lab Python Programing
EXPLORER
SKILL LAB PYTHON PROGRAMING
exp_9.py
Day_4 > exp_9.py > ...
# Find frequency of every signal
signal_frequency = Counter(signals)
print("Signal Frequencies:")
for signal, freq in signal_frequency.items():
print(f" {signal}: {freq}")
# Identify duplicate signal names
duplicates = {signal: freq for signal, freq in signal_frequency.items() if freq > 1}
print("\nDuplicate Signals:")
for signal, freq in duplicates.items():
print(f" {signal}: appears {freq} times")
# Extract signals containing "data"
data_signals = [signal for signal in signals if "data" in signal]
print("\nSignals containing 'data':", data_signals)
# Randomly select three signals
selected_signals = random.sample(list(set(signals)), 3)
print("\nRandomly selected signals for testing:", selected_signals)
# Create independent copy and modify
signals_copy = copy.deepcopy(signals)
signals_copy.append("test_clk") # Modify the copy
signals_copy[0] = "modified_clk"
print("\nOriginal signals:", signals)
print("Modified copy:", signals_copy)
print(f"Copy independent: {signals != signals_copy}")
# Calculate total number of unique signals
unique_signals = len(set(signals))
print(f"\nTotal unique signals: {unique_signals}")
import itertools
import random
frequencies = [10, 25, 50, 100, 200] # MHz
Ln 306, Col 43 Spaces: 4 UTF-8 CRLF Python Python 3.14 (64-bit) Go Live


File Edit Selection View Go Run Terminal Help
Skill Lab Python Programing
EXPLORER
SKILL LAB PYTHON PROGRAMING
Day_1
Day_2
Day_3
Day_4
Day_4.py
exp_7.py
exp_8.py
exp_9.py
HangManWindow.py
logic_testing.py
main.py
Experiments
test_data
git-cheat-sheet-education.pdf
invalid_samples.txt
signal_values.txt
test_configurations.txt
UserData.csv
users.csv
OUTLINE
TIMELINE
exp_9.py
Day_4 > exp_9.py > ...
Store selected configurations (in a file for persistence)
with open("test_configurations.txt", "w") as f:
for config in selected_configurations:
f.write(f"{config[0]},{config[1]},{config[2]},{config[3]}\n")
# Create an independent copy and modify it
config_copy = copy.deepcopy(selected_configurations)
# Modify the copy (e.g., change all voltages to 1.2V)
for i, config in enumerate(config_copy):
config_copy[i] = (config[0], 1.2, config[2], config[3])
print("\nOriginal first 3 configurations:")
for config in selected_configurations[:3]:
print(f" {config}")
print("\nModified copy first 3 configurations:")
for config in config_copy[:3]:
print(f" {config}")
# Verify independence
print(f"\nCopy independent: {selected_configurations != config_copy}")
from collections import Counter
import random
import copy
signals = [
"clk",
"reset",
"data_in",
"data_out",
"addr",
"addr",
"clk",
"valid",
"ready"
]
Ln 306, Col 43
Spaces: 4
UTF-8
CRLF
Python
Python 3.14 (64-bit)
Go Live

File Edit Selection View Go Run Terminal Help
Skill Lab Python Programing
EXPLORER
SKILL LAB PYTHON PROGRAMING
Day_1
Day_2
Day_3
Day_4
Day_4.py
exp_7.py
exp_8.py
exp_9.py
HangManWindow.py
logic_testing.py
main.py
Experiments
test_data
git-cheat-sheet-education.pdf
invalid_samples.txt
signal_values.txt
test_configurations.txt
UserData.csv
users.csv
OUTLINE
TIMELINE
exp_9.py
Day_4 > exp_9.py > ...
# Display summary
print("\n=== Verification Run Summary ===")
print(f"Total messages: {len(timestamps)}")
print(f"INFO messages: {type_counts.get('INFO', 0)}")
print(f"WARNING messages: {type_counts.get('WARNING', 0)}")
print(f"ERROR messages: {type_counts.get('ERROR', 0)}")
print(f"Total errors found: {len(error_messages)}")
print(f"Hex addresses referenced: {len(hex_addresses)}")
import itertools
import random
import copy
# Test parameters
clock_frequencies = ['100MHz', '200MHz', '400MHz']
voltage_levels = [0.9, 1.0, 1.1, 1.2]
operating_modes = ['normal', 'low_power']
temperature_conditions = ['25C', '85C', '125C']
# Generate all possible configurations using Cartesian product
all_configurations = list(itertools.product(
clock_frequencies,
voltage_levels,
operating_modes,
temperature_conditions
))
print(f"Total possible configurations: {len(all_configurations)}")
# Randomly select 10 configurations
selected_configurations = random.sample(all_configurations, 10)
print("\nSelected configurations for testing:")
for i, config in enumerate(selected_configurations, 1):
print(f" {i}. Frequency: {config[0]}, Voltage: {config[1]}V, "
f"Mode: {config[2]}, Temperature: {config[3]}")
Store selected configurations (in a file for persistence)
with open("test_configurations.txt", "w") as f:
Ln 306, Col 43
Spaces: 4
UTF-8
CRLF
Python
Python 3.14 (64-bit)
Go Live


File Edit Selection View Go Run Terminal Help
Skill Lab Python Programing
EXPLORER
SKILL LAB PYTHON PROGRAMING
Day_1
Day_2
Day_3
Day_4
Day_4.py
exp_7.py
exp_8.py
exp_9.py
HangManWindow.py
logic_testing.py
main.py
Experiments
test_data
git-cheat-sheet-education.pdf
invalid_samples.txt
signal_values.txt
test_configurations.txt
UserData.csv
users.csv
OUTLINE
TIMELINE
exp_9.py
Day_4 > exp_9.py > ...
import re
from collections import Counter
from datetime import datetime
# Sample log data (in practice, read from file)
log_content = """
[2026-08-12 10:15:21] INFO: Test started
[2026-08-12 10:15:25] ERROR: Address mismatch at 0x1A2F
[2026-08-12 10:15:28] WARNING: Timeout
[2026-08-12 10:15:30] ERROR: Data mismatch at 0x2B10
"""
# Extract timestamps
timestamps = re.findall(r'\[(\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\]', log_content)
print("Timestamps:", timestamps)
# Extract hexadecimal addresses
hex_addresses = re.findall(r'0x[0-9A-Fa-f]+', log_content)
print("Hex Addresses:", hex_addresses)
# Extract ERROR messages
error_messages = re.findall(r'ERROR: (.+)', log_content)
print("Error Messages:", error_messages)
# Count message types
message_types = re.findall(r'\] (INFO|WARNING|ERROR):', log_content)
type_counts = Counter(message_types)
print("\nMessage Type Counts:")
for msg_type, count in type_counts.items():
print(f" {msg_type}: {count}")
# Determine earliest and latest timestamps
datetime_objects = [datetime.strptime(ts, "%Y-%m-%d %H:%M:%S") for ts in timestamps]
earliest = min(datetime_objects)
latest = max(datetime_objects)
print(f"\nEarliest timestamp: {earliest}")
print(f"Latest timestamp: {latest}")
# Display summary
Ln 306, Col 43
Spaces: 4
UTF-8
CRLF
Python
Python 3.14 (64-bit)
Go Live

except FileNotFoundError:
# Create sample data for demonstration
signal_values = list(np.random.uniform(-0.5, 1.5, 1000))
with open("signal_values.txt", "w") as f:
for val in signal_values:
f.write(f"{val}\n")
# Convert to numpy array
signal_array = np.array(signal_values)
# Calculate statistics
min_val = np.min(signal_array)
max_val = np.max(signal_array)
mean_val = np.mean(signal_array)
std_val = np.std(signal_array)
rms_val = np.sqrt(np.mean(np.square(signal_array)))
print(f"Minimum: {min_val:.4f} V")
print(f"Maximum: {max_val:.4f} V")
print(f"Mean: {mean_val:.4f} V")
print(f"Standard Deviation: {std_val:.4f} V")
print(f"RMS Value: {rms_val:.4f} V")
# Identify values outside 0V to 1.2V range
invalid_mask = (signal_array < 0) | (signal_array > 1.2)
invalid_samples = signal_array[invalid_mask]
invalid_indices = np.where(invalid_mask)[0]
print(f"\nInvalid samples found: {len(invalid_samples)}")
# Store invalid samples in separate file
with open("invalid_samples.txt", "w") as f:
f.write("Index,Value\n")
for idx, val in zip(invalid_indices, invalid_samples):
f.write(f"{idx},{val:.4f}\n")
print("Invalid samples saved to invalid_samples.txt")


File Edit Selection View Go Run Terminal Help
Skill Lab Python Programing
EXPLORER
SKILL LAB PYTHON PROGRAMING
Day_1
Day_2
Day_3
Day_4
Day_4.py
exp_7.py
exp_8.py
exp_9.py
HangManWindow.py
logic_testing.py
main.py
Experiments
test_data
git-cheat-sheet-education.pdf
invalid_samples.txt
signal_values.txt
test_configurations.txt
UserData.csv
users.csv
OUTLINE
TIMELINE
exp_9.py
Day_4 > exp_9.py > ...
import random
import os
from collections import Counter
# Generate 100 random memory addresses in range 0x0000 to 0x0FFF
addresses = [random.randint(0x0000, 0x0FFF) for _ in range(100)]
# Identify duplicate addresses and display frequency
address_counts = Counter(addresses)
duplicates = {addr: count for addr, count in address_counts.items() if count > 1}
print("Duplicate Addresses:")
for addr, count in duplicates.items():
print(f" 0x{addr:04X}: appears {count} times")
# Randomly select 10 addresses for additional testing
selected_addresses = random.sample(addresses, 10)
print("\nSelected addresses for additional testing:")
for addr in selected_addresses:
print(f" 0x{addr:04X}")
# Create directory and save addresses
os.makedirs("test_data", exist_ok=True)
with open("test_data/addresses.txt", "w") as f:
for addr in addresses:
f.write(f"0x{addr:04X}\n")
print("\nAddresses saved to test_data/addresses.txt")
import numpy as np
# Read values from file (assuming file exists with one value per line)
try:
with open("signal_values.txt", "r") as f:
signal_values = [float(line.strip()) for line in f]
except FileNotFoundError:
# Create sample data for demonstration
signal_values = list(np.random.uniform(-0.5, 1.5, 1000))
Ln 306, Col 43
Spaces: 4
UTF-8
CRLF
Python
Python 3.14 (64-bit)
Go Live